

TECHNICAL DOCUMENT

# Unity 멀티플레이 아레나 PvP

Mirror + KCP 기반 2v2 팀 PvP 게임 — 서버 권위 구조, 데디케이트 서버 동적 할당,  
PC·모바일 단일 코드베이스

Unity

Mirror Networking

KCP2K

Node.js + MySQL

Linux

장르

3인칭 아레나 팀 PvP

플랫폼

PC · 모바일

개발 형태

개인 (1인 개발)

## 프로젝트 개요

Mirror 튜토리얼이 아닌, **실제 라이브 게임의 구조**를 그대로 구현한 멀티플레이 프로젝트.  
매치메이킹부터 데디 서버 동적 할당까지 처음부터 끝까지 직접 설계했습니다.

### GAME OVERVIEW

#### 브롤스타즈류 2v2 팀 아레나 PvP

3인칭 시점 · 4명 (2v2) · 180초 매치

|       |                |
|-------|----------------|
| 장르    | 3인칭 아레나 팀 PvP  |
| 플레이어  | 최대 4명 (2v2)    |
| 플랫폼   | PC · 모바일 동시 지원 |
| 개발 형태 | 개인 프로젝트 (1인)   |
| 핵심 기술 | Mirror + KCP2K |

### WHY THIS MATTERS

#### 이 프로젝트의 차별점

##### 01 상용 구조 그대로 구현

로비/배틀 서버 분리 · 포트 동적 할당  
실제 라이브 게임에서 사용하는 구조

##### 02 서버 권위 모델로 치트 차단

스탯 · 쿨다운 · 데미지 서버 재계산  
클라이언트 조작 원천 차단

##### 03 PC · 모바일 단일 코드베이스

IPlayerInputProvider 인터페이스  
플랫폼 분기 코드 0줄

#### 목표

상용 멀티플레이 게임의 서버 구조를 처음부터 끝까지 직접 설계하고 구현

#### 핵심 차별점

로비/배틀 서버 분리 · 포트 동적 할당 · 서버 권위 모델 · 단일 코드베이스

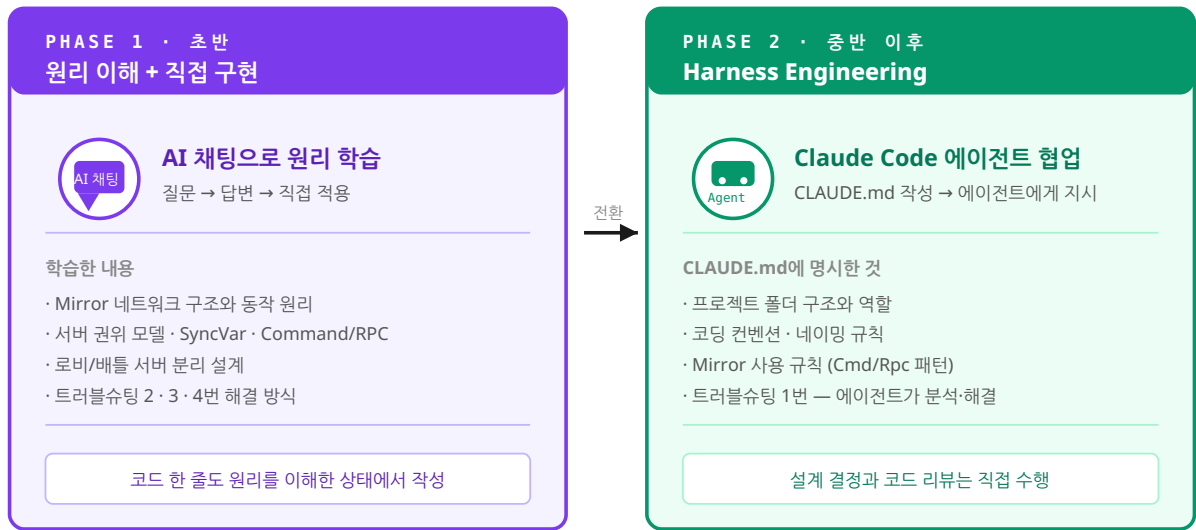
#### 결과물

실제 매치메이킹과 동시 다중 매치가 가능한 동작하는 멀티플레이 게임

## 개발 방식 — Harness Engineering

원리를 먼저 이해하고, 그 다음에 에이전트를 활용했습니다.

무엇을 시킬지 알아야 에이전트를 제대로 쓸 수 있기 때문입니다.



### 왜 이 순서인가

원리를 모르면 에이전트에게 무엇을 시킬지도 모름. 이해가 먼저, 위임이 나중

### HARNESS ENGINEERING

CLAUDE.md로 프로젝트 컨텍스트·규칙을 명시해 에이전트가 일관된 코드 생성

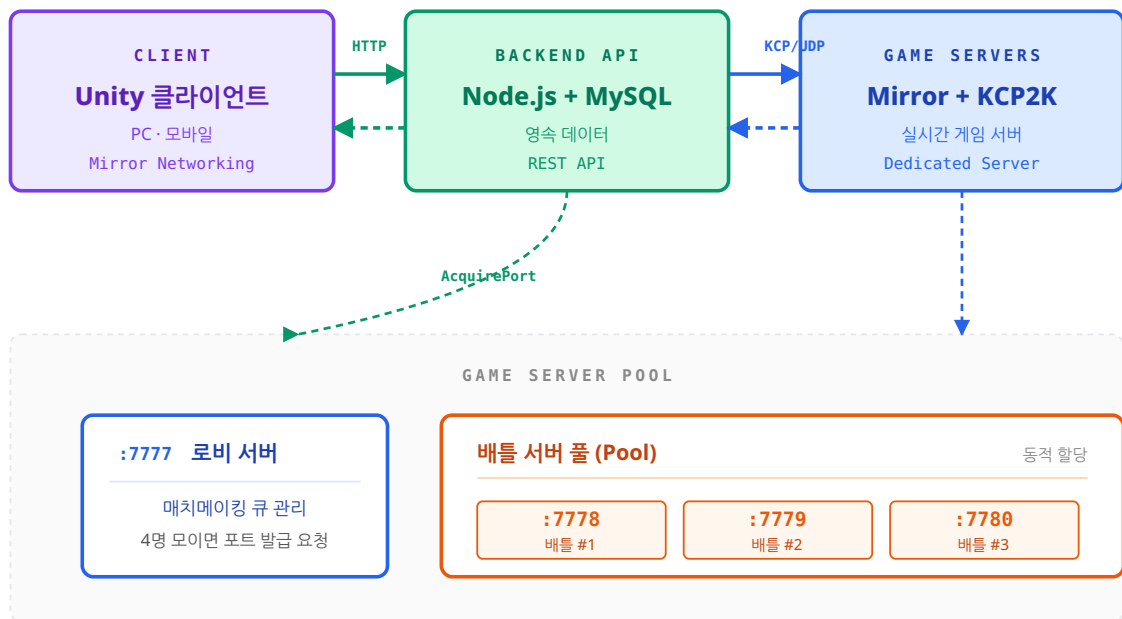
### 결과

1인 개발로 이 규모의 프로젝트를 완성. AI를 도구로 활용하는 능력 자체도 현대 개발 역량

## 시스템 아키텍처

**3-tier 구조**로 클라이언트 · 백엔드 API · 게임 서버를 분리.

영속 데이터는 HTTP REST로, 실시간 동기화는 KCP/UDP로 — 트래픽 특성에 맞춰 프로토콜을 나눴습니다.



### 클라이언트

Unity 단일 빌드로 PC·모바일  
동시 지원. Mirror로 게임 서버  
연결

### 백엔드 API

계정·인벤토리·가차·매치 결과 저장.  
POST /match/acquire 로 포트  
발급

### 게임 서버

로비 1개 + 배틀 3개. 모두  
Unity 배치모드로 실행되는 독립  
프로세스

## 동적 포트 할당

배틀 서버를 **포트 풀(7778~7780)**로 관리해 최대 3개 매치를 동시에 진행.  
백엔드가 사용 가능한 포트를 발급하고 매치 종료 시 자동 반환합니다.

### PROBLEM

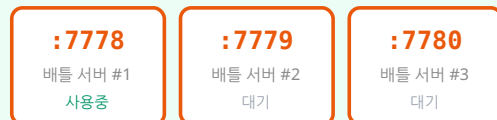
#### 단일 포트 사용 시

- X** 동시 매치 1개만 가능  
한 매치 진행 중이면 다른 4명은 대기
- X** 포트 충돌  
두 매치 동시 띄우면 같은 포트 점유 불가
- X** 확장 불가  
유저가 늘어나면 즉시 병목 발생

### SOLUTION

#### 포트 풀(Pool) 도입

##### PORT POOL



##### FLOW

- 4명 매치 성립
- AcquirePort 요청 → 백엔드
- 사용 가능한 포트 발급 (예: 7779)
- 매치 종료 시 ReleasePort로 반환

최대 3개 매치 동시 진행

#### 발급 API

```
POST /match/acquire →  
{ port: 7779 }
```

#### 반환 API

```
POST /match/release — 매치 종  
료 시 자동 호출
```

#### 확장 방법

유저 증가 시 풀 크기만 늘리면  
됨. 코드 변경 없이 수평 확장 가  
능

## 게임 흐름

유저는 **5개 씬**을 거치며, 각 씬은 독립된 NetworkManager 컨텍스트를 가집니다.  
로비 ↔ 배틀 전환 시 `StopClient` → 새 서버 `StartClient` 로 연결을 재맺습니다.



### 씬 분리 의도

각 씬이 한 가지 책임만. 로비는 매칭만, 배틀은 전투만 — 디버깅 용이

### 네트워크 컨텍스트

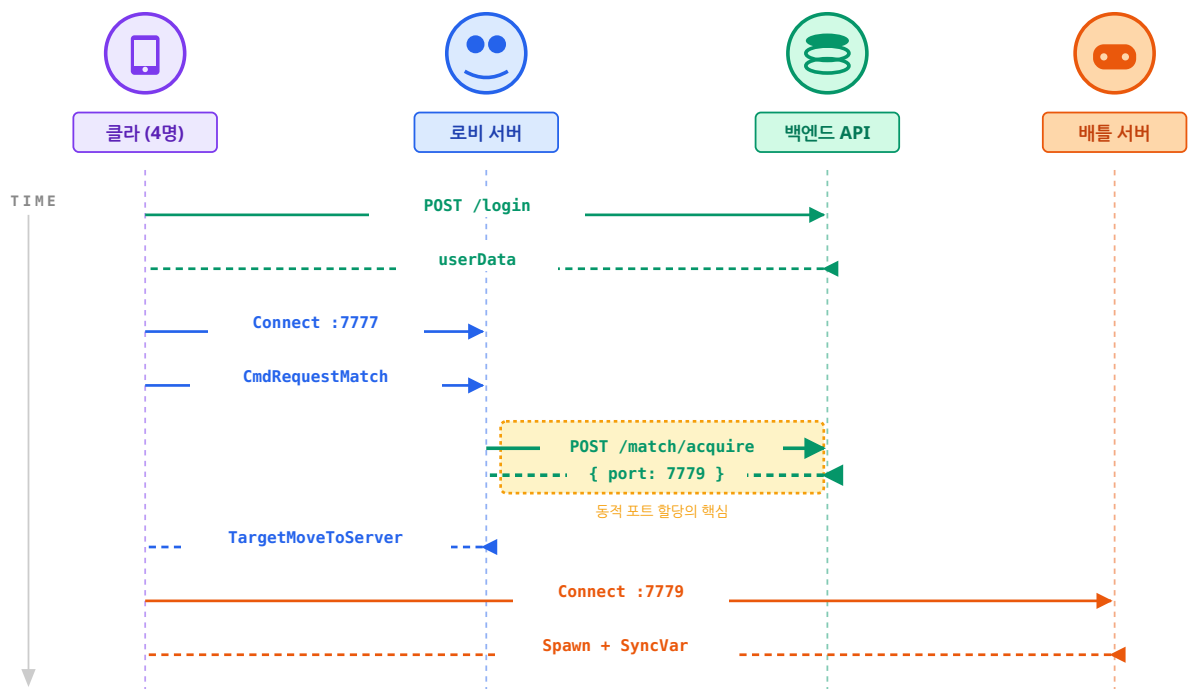
씬 전환마다 연결을 끊고 새로 맺어 각 씬의 네트워크 상태 독립 유지

### 메모리 효율

씬 전환 시 이전 씬 리소스 해제. 배틀 중에는 로비 UI가 메모리에 없음

## 통신 시퀀스

매칭 시점에 **클라이언트 ↔ 3종 서버(로비/백엔드/배틀)** 사이에서 9번의 메시지가 오갑니다.  
백엔드의 포트 발급이 핵심 — 이 단계가 동적 할당의 시작점입니다.



### HTTP (실선)

백엔드 API 요청/응답, 로그인 ·  
매치 포트 발급

### MIRROR CMD/RPC

로비 서버와의 매칭 · 서버 전환  
명령

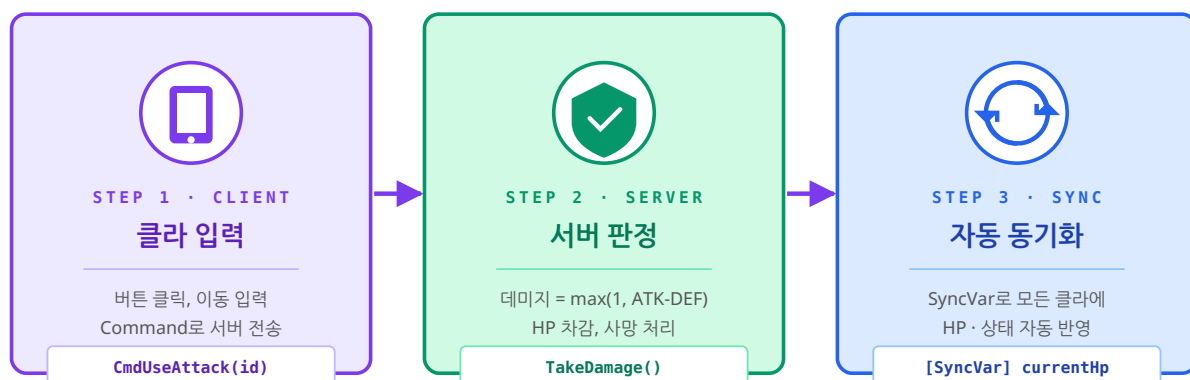
### 배틀 연결

매치 포트 4명 동시 접속, 스폰  
후 SyncVar로 상태 자동 동기화

## 서버 권위 모델

모든 게임 로직은 **서버에서만** 판정. 클라이언트는 입력 전달과 시각화만 담당.

`StatUtils.Calculate` 단일 진입점으로 클라/서버 동일 로직 보장 — 치트 차단 핵심입니다.



SYNCED DATA · 자동 동기화되는 데이터

|                           |                       |                       |                           |                               |
|---------------------------|-----------------------|-----------------------|---------------------------|-------------------------------|
| <b>currentHp</b><br>현재 체력 | <b>teamId</b><br>팀 번호 | <b>stats</b><br>최종 스탯 | <b>isStunned</b><br>기절 상태 | <b>selectedChar</b><br>선택 캐릭터 |
|---------------------------|-----------------------|-----------------------|---------------------------|-------------------------------|

### 단일 진입점

`StatUtils.Calculate` 를 클라/서버가 공유. 입력이 같으면 결과도 같음

### 검증 시점

접속 시 `MyAuthenticator` 가 스탯 재계산 → 불일치 시 연결 거부

### 동기화 방식

Mirror SyncVar — 서버에서 값 변경 시 모든 클라이언트에 자동 전파



## 전투 시스템

각 캐릭터는 3개 스킬 슬롯 보유. **SkillData**는 **List<SkillAction>**의 조합으로 구성되어 "Dash → Melee" 같은 복합 액션을 자유롭게 만들 수 있습니다.

### SKILL SLOTS

#### 3개의 스킬 슬롯



### COOLDOWN VERIFICATION

#### 쿨다운 이중 검증



### ATTACK TYPES & PROJECTILES



#### 스킬 조합

`SkillData(cooldown + List<SkillAction>)` 로 복합 액션 구성. 예: Dash → Melee

#### 이중 검증 효과

클라 쿨다운만 있으면 조작 가능.  
서버 쿨다운으로 실제 발동 차단

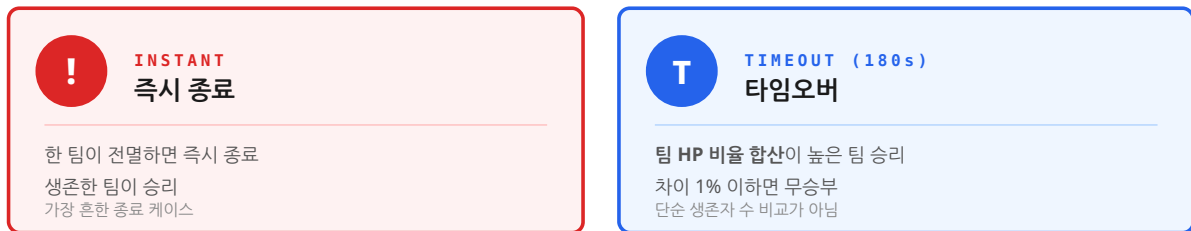
#### 입력 버퍼

0.15초 보존으로 쿨다운 직전 입력도 발동. 체감 반응성 크게 향상

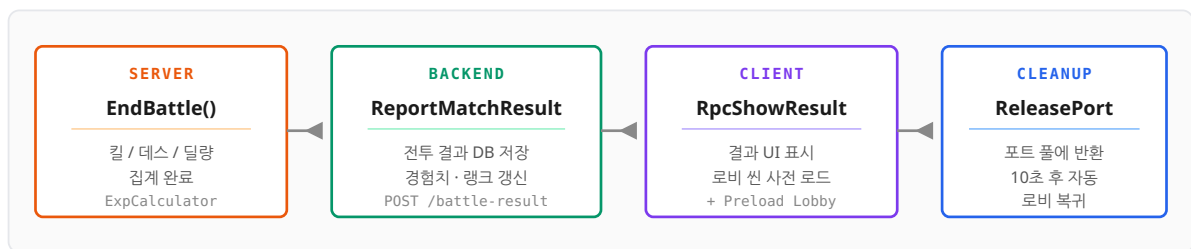
## 전투 판정 & 결과

180초 동안 **팀 전멸** 또는 **HP 비율 합산**으로 승패 결정.  
단순 생존자 수 비교가 아닌 HP 비율 방식으로 운빨 요소를 줄였습니다.

### VICTORY CONDITIONS · 승패 조건



### RESULT FLOW · 결과 전달 흐름



#### HP 비율 판정

팀 HP 비율 합산이 더 공정. 마지막까지 전략적으로 싸울 이유를 만듦

#### 집계 데이터

킬·데스·딜량을 서버가 실시간 추적. **ExpCalculator**로 경험치 산출

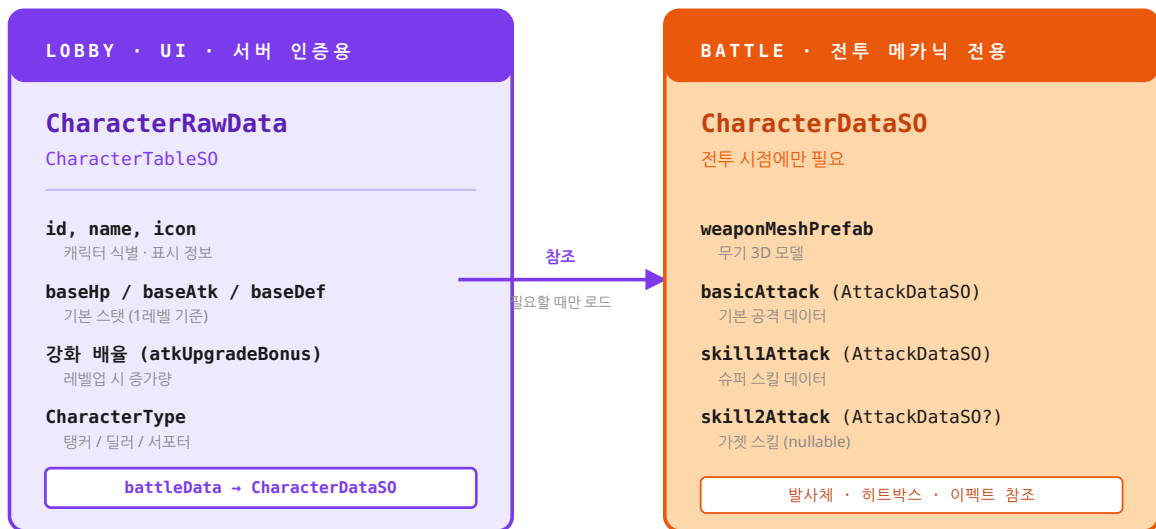
#### 자원 회수

매치 종료 → 포트 반환 → 다음 매치가 즉시 해당 포트 사용 가능

## 캐릭터 데이터 구조

ScriptableObject를 **로비용 / 전투용** 두 가지로 분리.

전투 데이터 변경이 로비 UI에 영향 없고, 컴파일 없이 밸런싱 작업이 가능합니다.



### 로비/UI 측 SO

자주 안 바뀜. 캐릭터 카드, 인벤토리, 스탯 표시에 사용

### 전투 메카닉 SO

자주 바뀜. 발사체·데미지·쿨다운 등 밸런싱 시 수정

### 분리 효과

전투 SO 수정 시 로비 UI 코드 영향 0. 컴파일 없이 밸런싱 가능

## 인벤토리 & 강화

드래그&드롭 장비 장착, 강화, 초월 기능 제공.

장비 변경 시 **StatUtils.Calculate**가 즉시 재계산되어 UI 스탯이 실시간 갱신됩니다.

### DATA FLOW · 데이터 흐름



### FEATURES · 주요 기능



#### DRAG & DROP 장착 / 해제

아이템 드래그&드롭으로  
캐릭터에 장비 장착

장착 변경 시 스탯 즉시 재계산  
**StatUtils.Calculate()**  
UI 실시간 갱신



#### ENHANCE 강화

캐릭터 레벨 상승  
기본 스탯 증가

강화 배율 적용  
**atkUpgradeBonus** 등  
EnhancePanel



#### TRANSCEND 초월

캐릭터 한계 돌파  
추가 스탯 부여

고급 진화 시스템  
**최상위 강화 단계**  
TranscendPanel

#### 실시간 반영

장착 변경 시 **StatUtils** 재계산  
→ UI 즉시 갱신. 서버 통신 없이  
미리보기

#### 서버 동기화

강화/초월 확정 시 백엔드 API  
호출 → DB 반영 → 결과 동기화

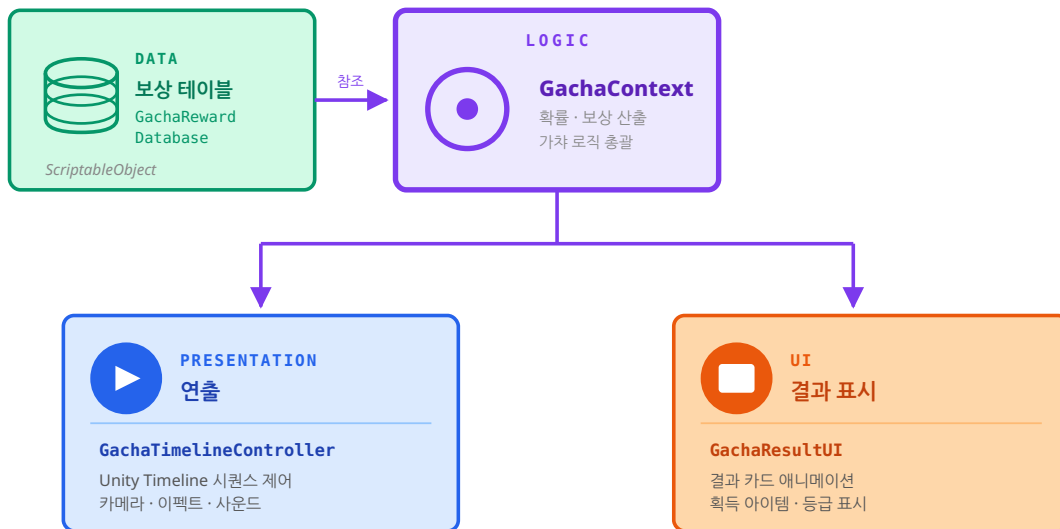
#### 데이터 모델

**PlayerInventory** 가 런타임 진  
실. **CharacterUIModel** 이 UI에 바  
인딩

## 가차 시스템

로직 · 연출 · UI를 명확히 분리한 4-컴포넌트 구조.

Unity Timeline으로 연출을 제어해 연출 변경이 로직에 영향을 주지 않습니다.



분리 효과: 연출 변경은 Timeline만, 보상 변경은 SO만, UI 변경은 Result만 수정 — 책임 명확

### 데이터

SO 기반 보상 테이블. 신규 보상 추가 시 코드 수정 없이 에디터에서 작업

### 로직

확률 계산과 보상 산출이 단일 클래스에 집중. 테스트 용이

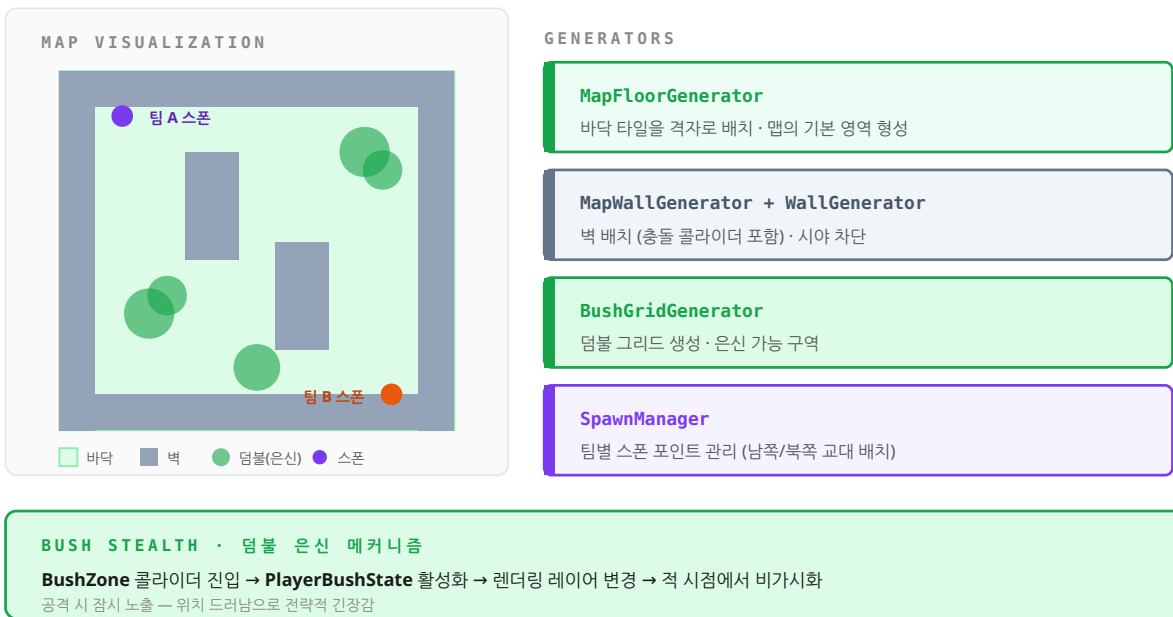
### 연출

Unity Timeline 사용. 디자이너가 코드 없이 연출 시퀀스 수정 가능

## 맵 생성 시스템

바닥 · 벽 · 덩불을 각각 **별도 Generator**가 절차적으로 생성.

덩불 안에 들어간 플레이어는 적에게 보이지 않는 **은신 시스템**이 작동합니다.



### 바닥 · 벽

절차적 생성으로 다양한 맵 구성 가능. 콜라이더로 충돌 처리

### 덩불 은신

**BushZone** + **PlayerBushState**  
— 전략적 위치 선점 요소

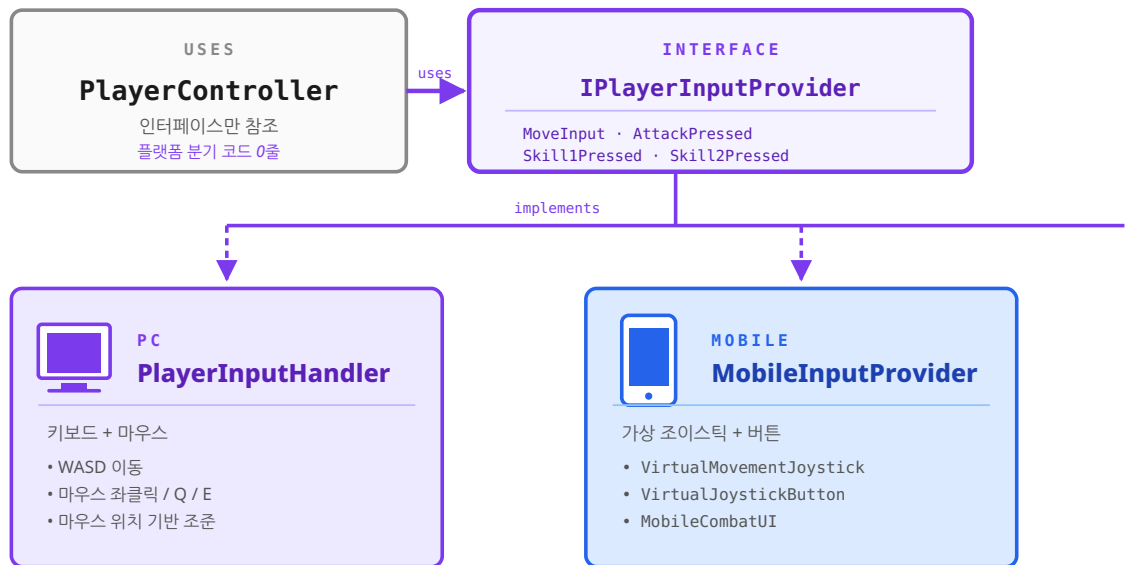
### 스폰

팀별 스폰 포인트 · 매치 후  
**ResetSpawnCounts()** 로 포트 재 사용

## 크로스 플랫폼

`IPlayerInputProvider` 인터페이스로 입력을 추상화.

`PlayerController`는 플랫폼을 모름 — 플랫폼별 분기 코드가 한 줄도 없습니다.



### 설계 원칙

DIP(Dependency Inversion Principle) — 구체가 아닌 추상에 의존

### 유지보수 효과

한 플랫폼 변경이 다른 플랫폼에 영향 없음. 회귀 버그 위험 차단

### 확장성

새 플랫폼(콘솔 등) 추가 시 인터페이스 구현만으로 대응 가능

## 트러블슈팅

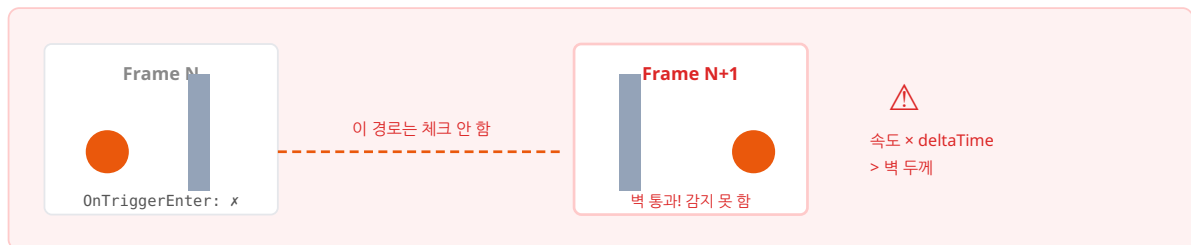
기술을 쓴다는 것과 **원리를 이해한다는 것**은 다릅니다.

아래 4가지 사례는 내부 동작 원리를 직접 파악하고 구조적으로 해결한 경험입니다.

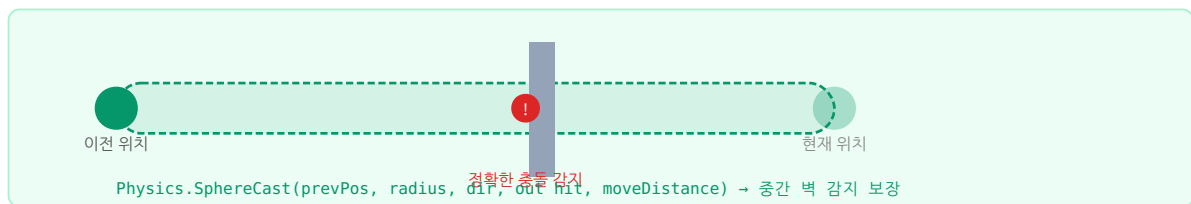
### 01 투사체 터널링

Claude Code 에이전트

**PROBLEM · Discrete 감지** - 경로 사이를 체크하지 않음



**SOLUTION · SphereCast** - 이전~현재 위치 전체 경로 스캔



#### 증상

빠른 투사체가 벽 통과. 디버그 로  
그로 `OnTriggerEnter` 가 아예 호  
출 안 됨을 확인

#### 원인 & 해결

Claude Code 에이전트가 코드  
베이스 분석 → 이산 감지 한계 파  
악 → SphereCast로 전체 경로  
스캔

#### 결과

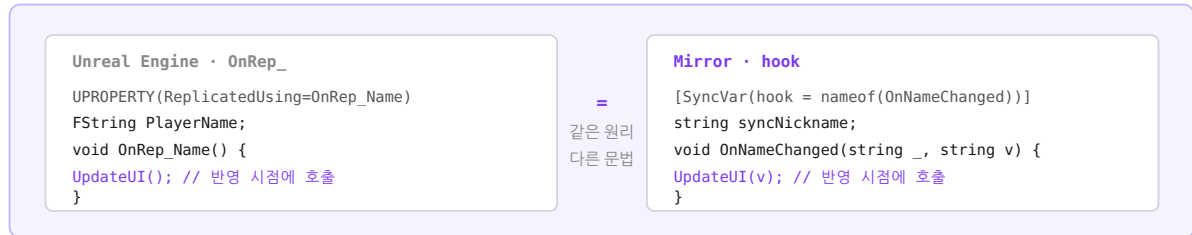
어떤 속도에서도 벽 관통 없음. 반  
사 투사체의 충돌 위치도 정확히  
계산



## PROBLEM · 스폰 직후 SyncVar 미보장



## SOLUTION · hook — 같은 원리, 다른 문법



## 증상

스폰 직후 닉네임·팀 색상이 빈값.  
 로그로 syncNickname 이 빈 문자열임을 확인

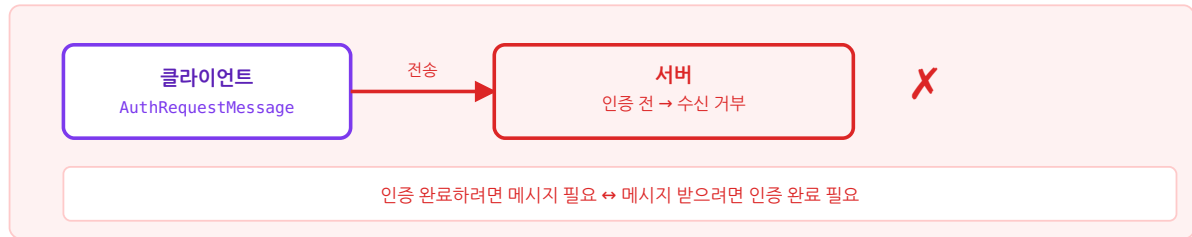
## 원인 &amp; 해결

언리얼 OnRep\_ 경험으로 Mirror hook의 필요성을 바로 파악. 값이 클라에 실제 반영되는 순간을 hook으로 잡음

## 결과

스폰 타이밍 문제 완전 해결. 두 엔진의 동기화 콜백이 같은 원리임을 확인

## PROBLEM · 순환 의존 구조

SOLUTION · `requireAuthentication = false` 옵션 추가

## 증상

클라이언트가 인증 메시지를 전송해도 서버 `OnAuthRequestMessage`가 호출되지 않음

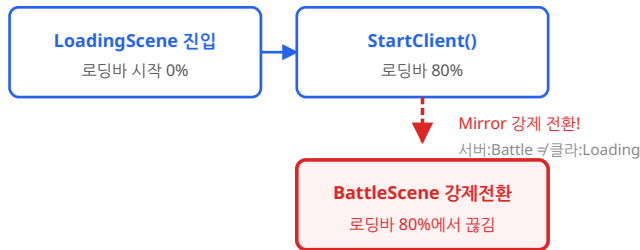
## 원인 &amp; 해결

인증 완료 전엔 메시지 수신 자체가 차단되는 순환 구조.  
`requireAuthentication = false` 옵션 하나로 해결

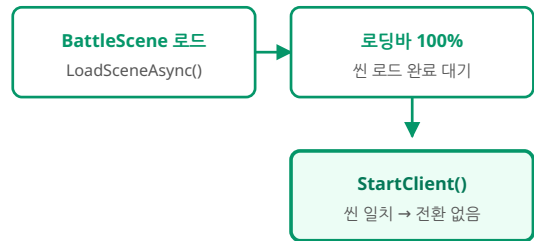
## 결과

인증 전 단계에서도 메시지 수신 가능. 이후 인증 흐름 정상 동작

## BEFORE · 문제 구조



## AFTER · 씬 먼저, 연결 나중



## MIRROR 씬 동기화 원리

Mirror는 **StartClient()** 시점에 서버 씬과 클라이언트 씬을 비교합니다.

서버 씬 < 클라이언트 씬 → onlineScene 설정 기반으로 클라이언트를 서버 씬으로 강제 전환

서버 씬 = 클라이언트 씬 → 추가 전환 없음, 연결만 수행

핵심: "씬 먼저 로드 → StartClient()" 순서로 Mirror 자동 전환을 우회

## 증상

로딩바 80%에서 갑자기 배틀씬 전환. 로그로 `onlineScene`에 의한 강제 전환 확인

## 원인 &amp; 해결

Mirror가 서버·클라 씬 불일치 시 자동 강제 전환한다는 원리 파악.  
씬 먼저 로드 → `StartClient()` 순서로 변경

## 결과

로딩바 100% 완료 후 씬 전환. 씬 전환과 서버 연결 타이밍을 완전히 제어

## 기술 하이라이트

단순 기능 구현이 아닌, **설계 결정이 담긴 7가지 핵심 포인트**.  
 각 항목은 "왜 그렇게 만들었는가"에 대한 답을 가집니다.



01

### 서버 권위 구조

스텝·콜다운 서버 이중  
검증 → 클라 조작 차단



02

### 로비/배틀 서버 분리

독립 프로세스 + 포트 폴  
다중 매치 동시 진행



03

### 입력 인터페이스 추상화

IPlayerInputProvider로  
PC/모바일 코드 통합



04

### 입력 버퍼 시스템

콜다운 직전 입력 0.15초  
보존 → 반응성 향상



05

### SO 데이터 분리

로비용/전투용 이원화  
컴파일 없이 밸런싱



06

### HP 비율 판정

단순 생존자 수 대신  
팀 HP 합산 → 공정 판정



07

### 절차적 맵 + 덤불 은신

맵 생성기 + BushZone  
전략적 공간 연출

#### CORE PRINCIPLE

"왜"가 명확한 설계. 각 결정은 구체적인 문제를 해결하기 위해 선택했으며, 모두 면접에서 설명할 수 있습니다.

## 프로젝트 구조

폴더는 **기능 단위**로 분리. 각 폴더가 명확한 책임을 가지며, 네이밍 규칙으로 역할을 즉시 식별할 수 있습니다.

### FOLDER STRUCTURE

Assets/\_dev/Script/

|              |                      |
|--------------|----------------------|
| └ Battle/    | # 전투 · 맵 · Bush      |
| └ Combat/    | # 무기 · 발사체           |
| └ Data/      | # ScriptableObject   |
| └ Gacha/     | # 가차 시스템             |
| └ Inventory/ | # 인벤토리 · 강화          |
| └ Lobby/     | # 로비 컨트롤러            |
| └ Login/     | # 로그인                |
| └ Player/    | # Controller · Stats |
| └ Server/    | # NetworkManager     |
| └ Shop/      | # 상점                 |
| └ UI/        | # 공통 UI              |

// 11개 기능 폴더 · 단일 책임 원칙

### NAMING CONVENTIONS

파일명만 봐도 역할 파악 가능

#### ~Manager

씬 또는 시스템 단위 매니저

BattleManager · SpawnManager

#### ~Controller

UI 또는 입력 컨트롤러

LobbyController · PlayerController

#### ~Data / ~SO

ScriptableObject 데이터

CharacterDataSO · AttackDataSO

#### Cmd~ / Rpc~

Mirror Command / RPC 메서드

CmdUseAttack · RpcShowDamage

## 끝까지 봐주셔서 감사합니다

프로젝트의 코드와 개발 과정이 궁금하시다면 아래 링크를 참고해주세요



G I T H U B

### 프로젝트 저장소

코드 · 커밋 히스토리

[github.com/ooweaJ/Unity\\_Dedicate](https://github.com/ooweaJ/Unity_Dedicate)



N O T I O N

### 개발일지

트러블슈팅 · 의사결정

[notion.so/Unity-Dedicated-Server](https://notion.so/Unity-Dedicated-Server)